PROJECT: RETAIL SALES ANALYTICS USING SNOWFLAKE

End-to-End Data Engineering Project

STEP-1: BUSINESS REQUIREMENT

Project Overview

This project demonstrates the implementation of an automated Retail Sales Analytics pipeline using Snowflake. Multiple retail stores submit daily sales files to Head Office. The pipeline ingests these files into Snowflake, captures data changes using Streams, processes incremental records using Tasks and MERGE operations, and maintains a curated reporting table.

Business Objective

- Automate daily sales data processing
- Handle Inserts, Updates, and Deletes
- Maintain a reporting-ready dataset
- Reduce manual intervention
- Enable reliable business reporting

Architecture Flow

Retail Stores
↓
Daily Sales Files
↓
Stage
↓
SALES_RAW
↓
STREAM
↓
TASK + MERGE
↓
SALES_CURATED
↓
Reports

STEP2 : PLATFORM SETUP

A dedicated Snowflake environment was created for the Retail Sales Analytics project to ensure logical separation from other development activities.

Database: RETAIL_SALES_DB

Schemas: RAW & CURATED

Warehouse: RETAIL_WH (XSMALL)

Purpose

- Isolate project objects
- Improve organization
- Simulate enterprise project structure
- Simplify maintenance and troubleshooting

((SQL script of platform setup is attached into RETAIL_SALES_OBJECTS.sql))

Warehouse created

```
SHOW WAREHOUSES LIKE 'RETAIL_WH';
```

esults (1 minute ago)

Table | Chart | Pivot

	name	state	type	size
1	RETAIL_WH	SUSPENDED	STANDARD	X-Small

Database created

```
USE DATABASE RETAIL_SALES_DB;
SELECT CURRENT_DATABASE();
```

ts (just now)

le | Chart | Pivot

CURRENT_DATABASE()
RETAIL_SALES_DB

Schemas created

```
SHOW SCHEMAS IN DATABASE RETAIL_SALES_DB;
```

esults (1 minute ago)

Table | Chart | Pivot

	created_on	name	is_default	is_current	database_name	owner
1	2026-09-14 00:48:52.217 -0700	CURATED	N	N	RETAIL_SALES_DB	ACCOUNTADMIN
2	2026-09-14 04:53:34.248 -0700	INFORMATION_SCHEMA	N	N	RETAIL_SALES_DB	
3	2026-09-14 00:48:40.777 -0700	PUBLIC	N	Y	RETAIL_SALES_DB	ACCOUNTADMIN
4	2026-09-14 00:48:48.853 -0700	RAW	N	N	RETAIL_SALES_DB	ACCOUNTADMIN

STEP 3: OBJECT CREATION

The core Snowflake objects required for the Retail Sales Analytics pipeline were created in the project environment. These objects collectively enable automated ingestion, change data capture, incremental processing, and reporting.

Objects Created

- File Format
  - Defines the structure of incoming CSV files.
- Internal Stage
  - Stores source sales files before loading into Snowflake tables.
- SALES_RAW
  - Landing table for incoming sales data.

- SALES_CURATED
    - Reporting-ready table containing the latest sales records.
- SALES_STREAM
    - Tracks changes occurring in the SALES_RAW table.
- MERGE SQL
    - Handles Insert, Update, Delete ingestion under Task into the CURATED.
- SALES_TASK
    - Automates incremental processing from RAW to CURATED.

MERGE & Task Processing

- SALES_STREAM captures changes from SALES_RAW.
- MERGE applies INSERT, UPDATE, and DELETE operations to SALES_CURATED.
- SALES_TASK executes the MERGE process automatically every minute.

Pipeline Flow

Sales File
↓
Stage
↓
SALES_RAW
↓
SALES_STREAM
↓
SALES_TASK
↓
MERGE
↓
SALES_CURATED

((SQL script of objects creation is attached into RETAIL_SALES_OBJECTS.sql))

All Objects Screenshots

File format created

```
14    SHOW FILE FORMATS LIKE 'SALES_CSV_FORMAT';
```

esults (just now)

Table | Chart | Pivot 1 row 69ms

	created_on	name	database_name	schema_name	type	owner	comment	format_options
1	2026-09-14 07:53:59	SALES_CSV_FORMAT	RETAIL_SALES_DB	RAW	CSV	ACCOUNTADMIN		{"TYPE":"CSV","RECOR

Stage created

```
SHOW STAGES LIKE 'SALES_STAGE';
```

esults (just now)

Table Chart Pivot

	created_on	name	database_name	schema_name
1	2026-09-14 07:54:02	SALES_STAGE	RETAIL_SALES_DB	RAW

Raw table created

```
SHOW TABLES LIKE 'SALES_RAW';
```

esults (just now)

Table Chart Pivot

	created_on	name	database_name	schema_name	kind
1	2026-09-14 07:54:28	SALES_RAW	RETAIL_SALES_DB	RAW	TABLE

Stream created

```
SHOW STREAMS LIKE 'SALES_STREAM';
```

esults (3 minutes ago)

Table Chart Pivot

	created_on	name	database_name	schema_name	kind
1	2026-09-14 07:54:28	SALES_RAW	RETAIL_SALES_DB	RAW	TABLE

Task created

```
SHOW TASKS LIKE 'SALES_TASK';
```

sults (just now)

Table Chart Pivot 1 row 182ms

	created_on	name	id	database_name	schema_name	owner	comment	warehouse	schedule
1	2026-09-14 08:02:53	SALES_TASK	01c71146-11e2-66d4	RETAIL_SALES_DB	RAW	ACCOUNTADMIN		RETAIL_WH	1 MINUTE

Curated table created

```
USE SCHEMA CURATED;
SHOW TABLES LIKE 'SALES_CURATED';
```

sults (just now)

Table Chart Pivot

	created_on	name	database_name	schema_name	kind
1	2026-09-14 07:54:52	SALES_CURATED	RETAIL_SALES_DB	CURATED	TABLE

STEP 4 - INITIAL DATA LOAD

A sales file containing 50 transactions was received from retail stores and loaded into the Snowflake pipeline.

Objective

- Ingest source sales data into the RAW layer

- Capture changes using Stream
- Process records into the CURATED layer using Task and MERGE

Data Load Summary

- Source File: day1_sales.csv
- Records Received: 50
- Load Status: Successful

Validation

- Data loaded into SALES_RAW
- Changes captured by SALES_STREAM
- SALES_TASK executed automatically
- Records merged into SALES_CURATED

Outcome

- RAW layer received all source records
- Stream captured the initial inserts
- Curated layer was populated successfully
- End-to-end pipeline functionality was verified

((SQL scripts of COPY INTO & Validation queries are attached into RETAIL_SALES_VALIDATION_QUERIES.sql))

Stage is loaded with the csv file

```
USE SCHEMA RAW;

LIST @SALES_STAGE;
```

sults (just now)

Table | Chart | Pivot

	name	size
1	sales_stage/day1_sales.csv	3472

Staged file data moved to the table

```
COPY INTO RAW.SALES_RAW
FROM @RAW.SALES_STAGE/day1_sales.csv
FILE_FORMAT = (FORMAT_NAME = SALES_CSV_FORMAT);
```

sults (just now)

Table | Chart | Pivot

	file	status	rows_parsed	rows_loaded
1	sales_stage/day1_sales	LOADED	50	50

Count of Raw table

```
--Total rows count
SELECT COUNT(*) AS RECORD_COUNT FROM RAW.SALES_RAW;
```

esults (just now)

Table | Chart | Pivot

	RECORD_COUNT
1	50

Store wise Raw table count

```
SELECT STORE_ID, COUNT(*) FROM RAW.SALES_RAW GROUP BY STORE_ID ORDER BY STORE_ID;
```

Table | Chart | Pivot — 5 rows — 116ms

STORE_ID	COUNT(*)
S001	10
S002	10
S003	10
S004	10
S005	10

Duplicates check

```
--Duplicates check
SELECT SALE_ID, COUNT(*) FROM RAW.SALES_RAW GROUP BY SALE_ID HAVING COUNT(*) > 1;
```

ust now

Table | Chart | Pivot

SALE_ID

Query produced no results

Stream count

```
SELECT COUNT(*) FROM RAW.SALES_STREAM;
```

sults (just now)

Table | Chart | Pivot

COUNT(*)
50

Stream status

```
SELECT METADATA$ACTION, COUNT(*) FROM RAW.SALES_STREAM GROUP BY METADATA$ACTION;
```

esults (just now)

Table | Chart | Pivot — 1 row — 249ms

METADATA$ACTION	COUNT(*)
INSERT	50

Task resumed

```sql
ALTER TASK SALES_TASK RESUME;
```

sults (just now)

Table | Chart | Pivot

	status
1	Statement executed successfully.

Curated table count

```sql
SELECT COUNT(*) FROM CURATED.SALES_CURATED;
```

esults (just now)

Table | Chart | Pivot

	COUNT(*)
1	50

Store wise Curated table count

```sql
SELECT STORE_ID, COUNT(*) FROM CURATED.SALES_CURATED GROUP BY STORE_ID ORDER BY STORE_ID;
```

sults (just now)

Table | Chart | Pivot | 5 rows | 311ms

	STORE_ID	COUNT(*)
1	S001	10
2	S002	10
3	S003	10
4	S004	10
5	S005	10

Raw & Curated counts comparison

```sql
--Compare Raw & Curated after Task processing
SELECT
    (SELECT COUNT(*) FROM RAW.SALES_RAW)      AS RAW_COUNT,
    (SELECT COUNT(*) FROM CURATED.SALES_CURATED) AS CURATED_COUNT;
```

sults (just now)

Table | Chart | Pivot | 1 row | 303ms

	RAW_COUNT	CURATED_COUNT
1	50	50

Stream status post Task execution

```sql
SELECT COUNT(*) AS RECORD_COUNT FROM RAW.SALES_STREAM;
```

esults (just now)

Table | Chart | Pivot

	RECORD_COUNT
1	0

STEP-5: DUPLICATE LOAD TEST

A duplicate load scenario was executed by reprocessing an already ingested sales file.

Objective

- This is to validate Snowflake file load behavior.
- And understand duplicate data impact on the pipeline.
- Also to verify the MERGE handling in the curated layer.

Observations

- The source file was reloaded using FORCE=TRUE.
- Duplicate records were inserted into SALES_RAW.
- Changes were captured by SALES_STREAM.
- SALES_TASK processed the changes automatically.
- SALES_CURATED retained the correct business view through MERGE processing.

Outcome

- RAW layer contained duplicate source records.
- Stream captured the duplicate load activity.
- Curated layer maintained expected record counts and data consistency.

((SQL scripts of COPY INTO & Validation queries are attached into RETAIL_SALES_VALIDATION_QUERIES.sql))

Attempting duplication scenario – Copying same day1 sales data to the Raw table forcefully

```
COPY INTO RAW.SALES_RAW
FROM @RAW.SALES_STAGE/day1_sales.csv
FILE_FORMAT = (FORMAT_NAME = SALES_CSV_FORMAT)
FORCE = TRUE;
```

esults (just now)

Table | Chart | Pivot

	file	status	rows_parsed	rows_loaded
1	sales_stage/day1_sale	LOADED	50	50

Duplicates loaded forcefully

```
SELECT COUNT(*) FROM RAW.SALES_RAW; --100
```

sults (just now)

Table | Chart | Pivot

	COUNT(*)
1	100

Due to Matched condition in Merge in Task, few fields are updated

for the matching sales ids

```
SELECT COUNT(*) FROM CURATED.SALES_CURATED; --old 50
```

esults (just now)

Table | Chart | Pivot

	# COUNT(*)
1	50

STEP-6: UPDATE & DELETE TEST

A change file containing new sales, corrected sales, and cancelled sales transactions was processed through the pipeline.

Changes Applied

- New sales records inserted
- Existing sales records updated
- Cancelled sales records removed

Validation

- Changes captured by SALES_STREAM
- SALES_TASK executed automatically
- MERGE synchronized SALES_CURATED
- Final curated dataset reflected the latest business state

((SQL scripts of COPY INTO & Validation queries are attached into RETAIL_SAELS_VALIDATION_QUERIES.sql))

Loading day2 file for corrections - update & delete – 10 rows

```
COPY INTO RAW.SALES_RAW
FROM @RAW.SALES_STAGE/day2_sales.csv
FILE_FORMAT = (FORMAT_NAME = SALES_CSV_FORMAT);
```

esults (just now)

Table | Chart | Pivot

	file	status	rows_parsed	rows_loaded
1	sales_stage/day2_sale	LOADED	10	10

New count of Raw table post day2 upload

```
SELECT COUNT(*) FROM RAW.SALES_RAW;
```

sults (just now)

Table | Chart | Pivot

	# COUNT(*)
	110

CDC

```
SELECT COUNT(*) FROM RAW.SALES_STREAM;
```

sults (just now)

Table | Chart | Pivot

	COUNT(*)
1	10

Curated count remain same even after day2 processed

```
SELECT COUNT(*) FROM CURATED.SALES_CURATED;
```

sults (just now)

Table | Chart | Pivot

	COUNT(*)
1	50

Store wise count in Curated post day2 upload

```
SELECT STORE_ID, COUNT(*) FROM CURATED.SALES_CURATED GROUP BY STORE_ID ORDER BY STORE_ID;
```

esults (just now)

Table | Chart | Pivot 5 rows 120ms

	STORE_ID	COUNT(*)
1	S001	11
2	S002	11
3	S003	11
4	S004	10
5	S005	7

STEP-7: REPORTING QUERIES

The curated sales data was queried to generate business-level reporting outputs. These queries demonstrate that the processed data is ready for downstream analytics and reporting.

Reports Created

- Store-wise Sales Summary
- Product-wise Sales Summary
- Daily Sales Summary

Store-wise Sales

```
SELECT STORE_ID, COUNT(*) AS SALES_COUNT, SUM(AMOUNT) AS TOTAL_SALES FROM CURATED.SALES_CURATED GROUP BY STORE_ID ORDER BY STORE_ID;
```

esults (just now)

Table | Chart | Pivot 5 rows 201ms

	STORE_ID	SALES_COUNT	TOTAL_SALES
1	S001	11	8600.00
2	S002	11	6350.00
3	S003	11	9400.00
4	S004	10	9450.00
5	S005	7	10100.00

Product-wise Sales

```
SELECT PRODUCT_ID, COUNT(*) AS SALES_COUNT, SUM(AMOUNT) AS TOTAL_SALES FROM CURATED.SALES_CURATED GROUP BY PRODUCT_ID ORDER BY PRODUCT_ID;
```

sults (just now)

Table | Chart | Pivot — 10 rows — 333ms

	PRODUCT_ID	SALES_COUNT	TOTAL_SALES
	P501 10.0% P502 10.0% +8 more	2 – 6	2450 – 7650
1	P501	6	3500.00
2	P502	6	3600.00
3	P503	6	5400.00
4	P504	5	4500.00
5	P505	2	2450.00
6	P506	5	5100.00
7	P507	5	2750.00
8	P508	5	4000.00
9	P509	5	4950.00
0	P510	5	7650.00

Daily Sales Summary

```
SELECT SALE_DATE, COUNT(*) AS SALES_COUNT, SUM(QUANTITY) AS TOTAL_QUANTITY, SUM(AMOUNT) AS TOTAL_SALES FROM CURATED.SALES_CURATED GROUP BY SALE_DATE ORDER BY SALE_DATE;
```

esults (just now)

Table | Chart | Pivot — 2 rows — 442ms

	SALE_DATE	SALES_COUNT	TOTAL_QUANTITY	TOTAL_SALES
1	2026-09-01	43	107	34300.00
2	2026-09-02	7	25	9600.00

STEP-8: PROJECT SUMMARY & KEY LEARNINGS

Project Summary

The Retail Sales Analytics project implemented an end-to-end incremental data pipeline in Snowflake for processing daily sales data from multiple retail stores.

Key Snowflake Features Used

- Internal Stage and File Format
- RAW and CURATED data layers
- Streams for change tracking
- Tasks for automated processing
- MERGE for INSERT, UPDATE & DELETE handling
- Duplicate file-load handling
- Reporting-ready queries

Key Learnings

- Built a complete pipeline, from File → Stage → RAW → Stream → Task/MERGE → CURATED pipeline.
- Handled initial, duplicate, updated and cancelled sales records.
- Validated data at RAW, Stream and CURATED layers.
- Understood Snowflake's file-load history and controlled duplicate reprocessing using FORCE=TRUE.
- Maintained the latest business-ready data in the CURATED layer.

Future Enhancements

- Snowpipe-based automated ingestion

- External cloud storage integration
- JSON/Parquet source formats
- Larger datasets for performance testing
- BI/dashboard integration

Outcome

The project successfully demonstrated a practical **Snowflake incremental data pipeline** from source-file ingestion through reporting-ready data.